



UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"
Campus de São José dos Campos
Instituto de Ciência e Tecnologia

Non-linear Classifiers

Generalized Linear Classifiers, RBF Networks, SVM with Kernel Functions,
and CART

Prof. Dr. Rogério Galante Negri

Agenda

1 "Highly" non-linear problems

2 Generalized Linear Classifiers

3 RBF Networks

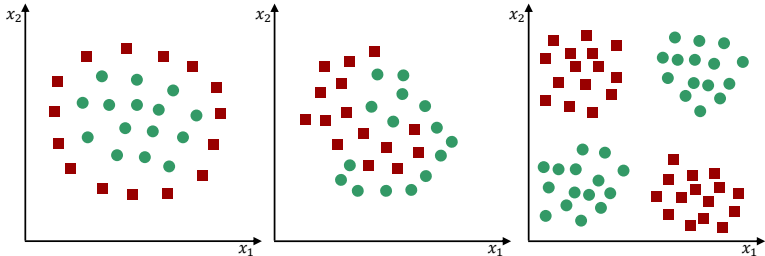
4 SVM and Kernel Functions

5 Decision Trees

6 Computational Experiment

"Highly" non-linear problems

- One way to handle this kind of problem is through the use of **non-linear classifiers**
- Remapping the original data to a new feature space is an effective approach



Agenda

1 "Highly" non-linear problems

2 Generalized Linear Classifiers

3 RBF Networks

4 SVM and Kernel Functions

5 Decision Trees

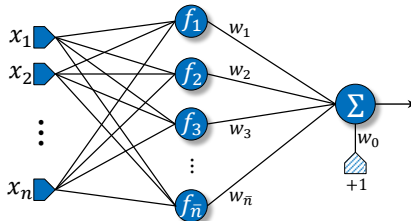
6 Computational Experiment

Generalized Linear Classifiers

- Let $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^n$ be a pattern from a non-linearly separable classification problem
- With functions $f_j: \mathbb{R}^n \rightarrow \mathbb{R}; j = 1, \dots, \bar{n}$, a remapping is defined as $\mathbf{x} \mapsto \mathbf{z} = [f_1(\mathbf{x}), \dots, f_{\bar{n}}(\mathbf{x})]$
- We can define a new space $\mathcal{Z} \subseteq \mathbb{R}^{\bar{n}}$ where the problem becomes linearly separable!

A nonlinear surface emerges

$$g(\mathbf{z}) = \mathbf{w}^T \mathbf{z} + w_0 > (<) 0 \Leftrightarrow g(\mathbf{x}) = \left[\sum_{j=1}^{\bar{n}} w_j f_j(\mathbf{x}) \right] + w_0 > (<) 0 \quad (1)$$



Cover's Theorem... the Number of Possibilities

The remapping process from a space of dimension n to $\bar{n}$, with $n < \bar{n}$, is supported by an important result known as [Cover's Theorem](#).

Theorem (Cover, 1965):

A complex pattern classification problem, with patterns non-linearly arranged in a high-dimensional space, is more likely to be linearly separable than in a low-dimensional space.

- Number of linearly separable dichotomies with m patterns in a space of dimension n :

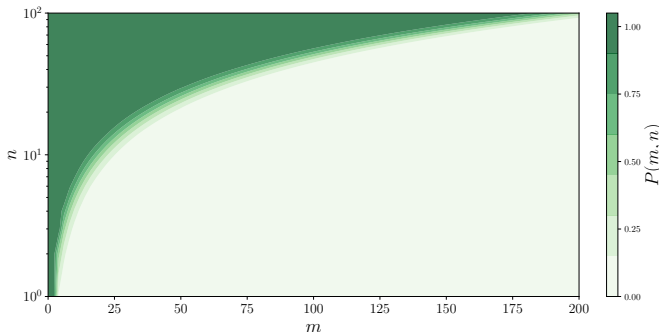
$$D(m, n) = \begin{cases} 2 \sum_{j=0}^{n-1} \binom{m-1}{j}; & m > n + 1 \\ 2^m; & m \leq n + 1 \end{cases} \quad (2)$$

- $j = 1, \dots, n - 1$ corresponds to the different degrees of freedom the linear separator can assume in n -dimensional space
- It is assumed that the m patterns occupy "general positions"

Cover's Theorem... and the Probability of Success

- Considering that there are 2^m dichotomies (whether linearly separable or not), when randomly selecting one of them, the probability that it is linearly separable is given by:

$$P(m, n) = \frac{D(m, n)}{2^m} = \begin{cases} 2^{m-1} \sum_{j=0}^{n-1} \binom{m-1}{j}; & m > n + 1 \\ 1; & m \leq n + 1 \end{cases} \quad (3)$$



Agenda

1 "Highly" non-linear problems

2 Generalized Linear Classifiers

3 RBF Networks

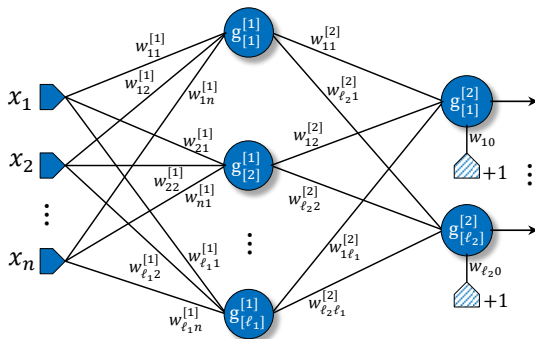
4 SVM and Kernel Functions

5 Decision Trees

6 Computational Experiment

RBF Networks

- RBF (Radial Basis Function) Networks are a type of neural network designed for classification and regression problems
- Characteristic architecture!



Activation Functions in RBF Networks

- The neurons located in the hidden layer have Gaussian-type activation functions:

$$g_{[j]}^{[1]}(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{w}_{[j]}^{[1]}\|^2}{2\sigma_{[j]}^2}\right) \quad (4)$$

with $j = 1, \dots, \ell_1$

- $\sigma_{[j]}$ corresponds to a parameter associated with the j -th neuron
- The activation functions of the neurons in the output layer are (typically) linear, that is:

$$g_{[k]}^{[2]}(\mathbf{z}_{[j]}^{[1]}) = \mathbf{w}_{[k]}^{[2]T} \mathbf{z}_{[j]}^{[1]} + w_{k0} \quad (5)$$

with $k = 1, \dots, \ell_2$

Second Phase (Supervised)

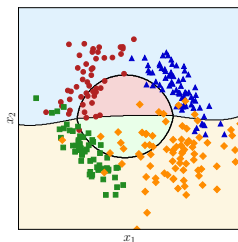
We can use the same procedure adopted by the SSE method

Algebraic modeling	$g_{pq} = g_{[q]}^{[1]}(\mathbf{x}_p)$	$y_{pq} = 1$ if $(\mathbf{x}_p, \omega_q)$
$\mathbf{G} = \begin{bmatrix} g_{11} & g_{12} & \cdots & g_{1\ell_1} \\ g_{21} & g_{22} & \cdots & g_{2\ell_1} \\ \vdots & \vdots & \ddots & \vdots \\ g_{m1} & g_{m2} & \cdots & g_{m\ell_1} \end{bmatrix}$	$\mathbf{W} = \begin{bmatrix} w_{11}^{[2]} & w_{12}^{[2]} & \cdots & w_{1\ell_1}^{[2]} \\ w_{21}^{[2]} & w_{22}^{[2]} & \cdots & w_{2\ell_1}^{[2]} \\ \vdots & \vdots & \ddots & \vdots \\ w_{\ell_2 1}^{[2]} & w_{\ell_2 2}^{[2]} & \cdots & w_{\ell_2 \ell_1}^{[2]} \end{bmatrix}$	
$\mathbf{Y} = \begin{bmatrix} y_{11} & y_{12} & \cdots & y_{1\ell_2} \\ y_{21} & y_{22} & \cdots & y_{2\ell_2} \\ \vdots & \vdots & \ddots & \vdots \\ y_{m1} & y_{m2} & \cdots & y_{m\ell_2} \end{bmatrix}$		

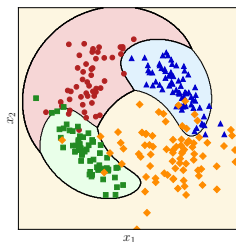
Once this matrix representation is established, the behavior of the RBF network can be expressed as $\mathbf{GW}^T = \mathbf{Y}$, thus:

$$\begin{aligned} \mathbf{GW}^T = \mathbf{Y} &\Leftrightarrow \mathbf{G}^T \mathbf{GW}^T = \mathbf{G}^T \mathbf{Y} \Leftrightarrow \\ &\Leftrightarrow (\mathbf{G}^T \mathbf{G})^{-1} (\mathbf{G}^T \mathbf{G}) \mathbf{W}^T = (\mathbf{G}^T \mathbf{G})^{-1} \mathbf{G}^T \mathbf{Y} \Leftrightarrow \mathbf{W}^T = (\mathbf{G}^T \mathbf{G})^{-1} \mathbf{G}^T \mathbf{Y} \end{aligned}$$

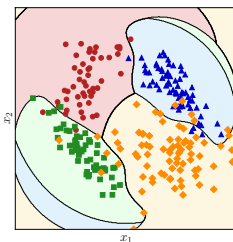
Classification vs. Number of Hidden Neurons



(a) $\ell_1 = 3$.



(b) $\ell_1 = 5$.



(c) $\ell_1 = 20$.

Figure: Decision surfaces obtained by RBF networks with different numbers of hidden neurons on a multiclass problem.

The adjustment capability of the decision surface increases as the number of neurons grows — [an effect derived from Cover's Theorem](#)

Agenda

1 "Highly" non-linear problems

2 Generalized Linear Classifiers

3 RBF Networks

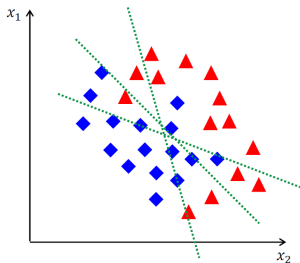
4 SVM and Kernel Functions

5 Decision Trees

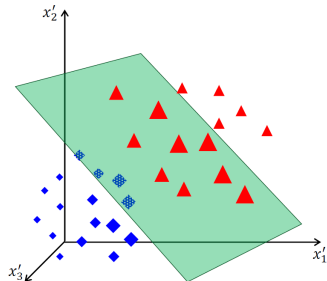
6 Computational Experiment

Support Vector Machines and Kernel Functions

- SVM — separation hyperplanes — **not always sufficient!**
- The alternative is to remap the patterns to another space



(a) Patterns distributed in the *feature space* in a non-linearly separable way. Using hyperplanes does not provide adequate separation



(b) Mapping of the patterns into a higher-dimensional *feature space*, where linear separation is possible

Figura: Example of mapping patterns to a higher-dimensional space that enables linear separation.

Kernel Functions

- The mapping process can be conducted implicitly
- To do this, simply replace the inner products $\mathbf{w}^T \mathbf{x}_i$ and $\mathbf{x}_i^T \mathbf{x}_j$ present in the SVM decision rule and objective function with a function $K(\cdot, \cdot)$ that satisfies the conditions of *Mercer's Theorem*

Mercer's Theorem:

Let $\mathbf{x}, \mathbf{z} \in \mathcal{X} \subseteq \mathbb{R}^n$ and $\phi(\cdot)$ be a mapping $\mathbf{x} \mapsto \phi(\mathbf{x}) \in \mathcal{H}$, with $\mathcal{H}$ being a Hilbert space. The inner product $\phi(\mathbf{x})^T \phi(\mathbf{z})$ is equivalent to the continuous and symmetric function $K(\mathbf{x}, \mathbf{z})$ that satisfies:

$$\int_{\mathcal{X}} \int_{\mathcal{X}} K(\mathbf{x}, \mathbf{z}) h(\mathbf{x}) h(\mathbf{z}) d\mathbf{x} d\mathbf{z} \geq 0, \quad (6)$$

for any $h(\mathbf{x})$ such that:

$$\int_{\mathcal{X}} h(\mathbf{x})^2 d\mathbf{x} < +\infty. \quad (7)$$

Kernel Functions

- Although Mercer's Theorem provides necessary conditions for constructing functions that define the inner product in some vector space, it does not indicate what that space is
- ...in other words, it does not provide a direct way to define $\phi(\cdot)$ from $K(\cdot, \cdot)$
- Verifying whether a given function $K(\cdot, \cdot)$ satisfies Mercer's Theorem conditions may not be trivial

Common Kernel Functions

$$K_{\text{RBF}}(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2}, \quad (8)$$

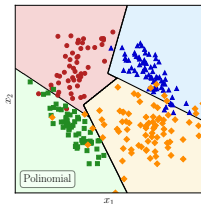
$$K_{\text{Pol}}(\mathbf{x}_i, \mathbf{x}_j) = \left(\gamma \mathbf{x}_i^T \mathbf{x}_j + \alpha \right)^q, \quad (9)$$

$$K_{\text{Sigm}}(\mathbf{x}_i, \mathbf{x}_j) = \tanh \left(\gamma \mathbf{x}_i^T \mathbf{x}_j + \alpha \right), \quad (10)$$

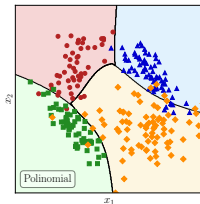
$$K_{\text{Linear}}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j, \quad (11)$$

where $q \in \mathbb{N}^*$ and $\gamma, \alpha \in \mathbb{R}_+^*$ are associated parameters

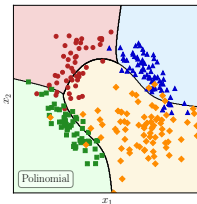
Classification vs. Kernels



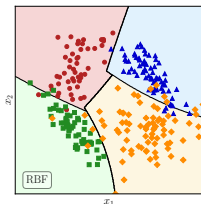
(a) Poly. - $q = 1$.



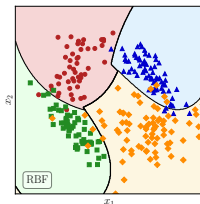
(b) Poly. - $q = 3$.



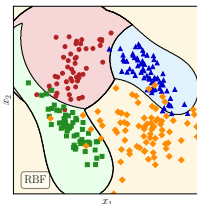
(c) Poly. - $q = 5$.



(d) RBF - $\gamma = 0.1$.

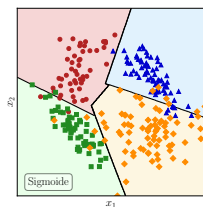


(e) RBF - $\gamma = 0.5$.

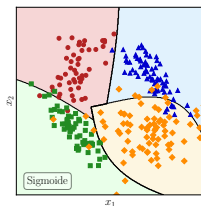


(f) RBF - $\gamma = 1.5$.

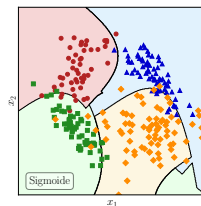
Classification vs. Kernels (cont.1)



(a) Sigmoid - $\gamma = 0.1$,
 $\alpha = 0$.



(b) Sigmoid - $\gamma = 0.5$,
 $\alpha = 0$.



(c) Sigmoid - $\gamma = 1.5$,
 $\alpha = 0$.

- Decision surfaces obtained using the SVM method with different kernel functions and parameter settings on a multiclass problem. The OVR strategy and penalty parameter $C = 100$ were used.
- In addition to the kernel function, the chosen parameters also influence the final result!

Agenda

1 "Highly" non-linear problems

2 Generalized Linear Classifiers

3 RBF Networks

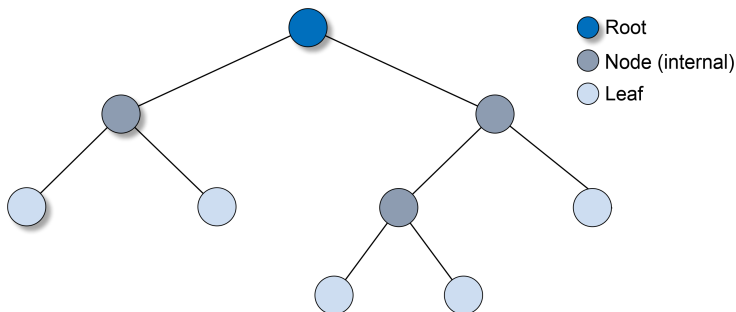
4 SVM and Kernel Functions

5 Decision Trees

6 Computational Experiment

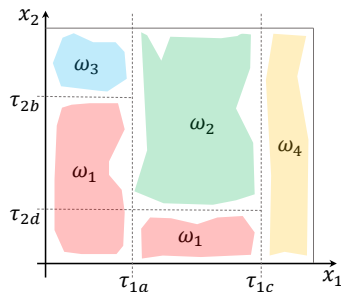
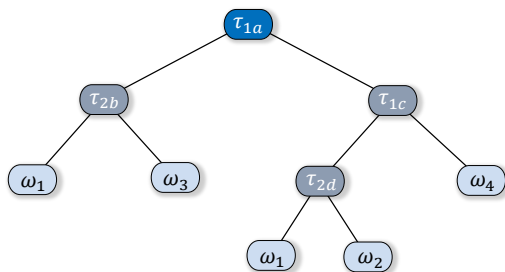
Decision Trees

- A binary tree is structured through vertices (or **nodes**) and hierarchical relationships
- Starting from a node, usually called the **root**, successive binary branches can be defined, which generate new nodes
- A node is said to be **internal** if it generates branches; otherwise, it is called a **leaf**



Decision Trees

- This structure provides a distinct non-linear classification approach
- It is based on successive checks of attributes from the training data
- A **threshold** is determined on **one of the attributes** of the samples, which then allows splitting the examples into two subsets
- It aims to **minimize variability**



CART (*Classification and Regression Trees*)

- Given the training set $\mathcal{D} = \{(\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, m\}$, where $\mathcal{X} \subseteq \mathbb{R}^n$ and $\mathcal{Y} = \{1, 2, \dots, c\}$
- Initially, let $\mathcal{Q} \equiv \mathcal{D}$
- Let $\mathcal{Q}_k = \{x_k : (\mathbf{x}, y) \in \mathcal{Q}; \mathbf{x} = [x_1, \dots, x_k, \dots, x_n]\}$ be the set of observed (non-repeated) values for the k -th attribute of the examples in $\mathcal{Q}$
- We can obtain values τ_{kh} , for $k = 1, \dots, n$ and $h = 1, \dots, \#\mathcal{Q}_k$, which define $\mathcal{Q}_{inf}(\tau_{kh})$ and $\mathcal{Q}_{sup}(\tau_{kh})$ as subsets of $\mathcal{Q}$ where the k -th attribute of $\mathbf{x}$ is less than or greater-than-or-equal to τ_{kh} , respectively

$$\mathcal{Q}_{inf}(\tau_{kh}) = \{(\mathbf{x}, y) \in \mathcal{Q} : x_k < \tau_{kh}\}$$

$$\mathcal{Q}_{sup}(\tau_{kh}) = \{(\mathbf{x}, y) \in \mathcal{Q} : x_k \geq \tau_{kh}\}$$

CART (*Classification and Regression Trees* (cont.1))

- The class variability within $\mathcal{Q}$, also known as **impurity**, is usually measured using the **Information Entropy**

$$I(\mathcal{Q}) = - \sum_{j=1}^c P(\omega_j | \mathcal{Q}) \log_2 P(\omega_j | \mathcal{Q}), \quad (12)$$

where:

$$P(\omega_j | \mathcal{Q}) = \frac{1}{\#\mathcal{Q}} \sum_{i=1}^{\#\mathcal{Q}} \delta_j(y_i),$$
$$\delta_j(y_i) = \begin{cases} 1; & \text{if } y_i = j, \\ 0; & \text{otherwise.} \end{cases}$$

CART (*Classification and Regression Trees* (cont.2))

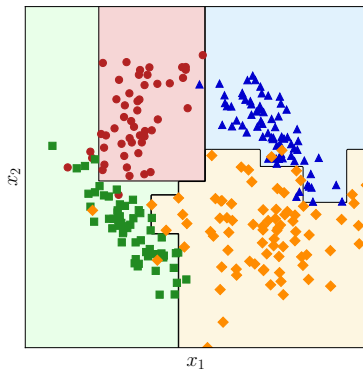
- Given a threshold τ_{kh} , it is possible to measure the impurity reduction achieved by splitting $\mathcal{Q}$ into $\mathcal{Q}_{inf}(\tau_{kh})$ and $\mathcal{Q}_{sup}(\tau_{kh})$

$$\Delta I(\mathcal{Q}; \tau_{kh}) = I(\mathcal{Q}) - \frac{\#\mathcal{Q}_{inf}(\tau_{kh})}{\#\mathcal{Q}} I(\mathcal{Q}_{inf}(\tau_{kh})) - \frac{\#\mathcal{Q}_{sup}(\tau_{kh})}{\#\mathcal{Q}} I(\mathcal{Q}_{sup}(\tau_{kh})) \quad (13)$$

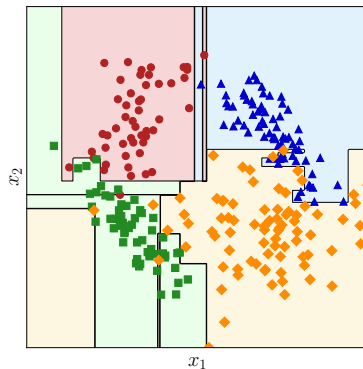
- When $\Delta I(\mathcal{Q}; \tau_{kh})$ exceeds a predefined threshold $\zeta \in \mathbb{R}_+$
- ...and the number of elements in $\mathcal{Q}$ also exceeds a minimum value $\psi \in \mathbb{N}^*$, the split is considered justified
- All previously discussed processes are conducted **recursively** on each resulting subset
- If the split is not justified, the node associated with the (local) set $\mathcal{Q}$ is labeled as a **leaf**, and must represent one of the classes $\omega^* \in \Omega$

$$\omega^* = \arg \max_{\omega_j \in \Omega} P(\omega_j | \mathcal{Q}). \quad (14)$$

Classification vs. "leaf size"



(d) $\psi = 10$.



(e) $\psi = 2$.

Figura: Decision surface obtained by the CART method on a multiclass problem. The parameter ζ , which defines the minimum impurity reduction, is set to 10^{-7} for all cases.

Agenda

1 "Highly" non-linear problems

2 Generalized Linear Classifiers

3 RBF Networks

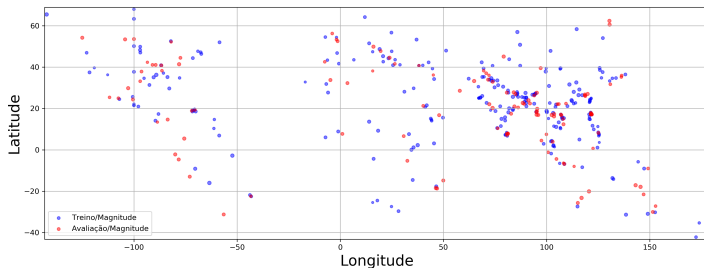
4 SVM and Kernel Functions

5 Decision Trees

6 Computational Experiment

Computational Experiment

- Prediction of flood-causing phenomena — flood occurrence data from different regions of the world
- Data provided by the Dartmouth Flood Observatory
- Classes — *heavy rain, ice melt, torrential rain, monsoon rain, and others*
- Attributes — *severity, longitude, latitude, and magnitude*



Computational Experiment (cont.1)

The MLP, RBF networks, SVM, and CART methods, under different parameter configurations, are trained and their generalization capabilities are assessed based on the percentage of correct predictions made on evaluation data.

Tabela: Accuracy values obtained by the analyzed methods.

Method	Configuration	Accuracy
MLP	Neurons/Layers	6 45.66%
		3-3 37.01%
		2-2-2 27.56%
RBF Network	Hidden neurons	$\ell_1 = 3$ 40.16%
		$\ell_1 = 5$ 37.01%
		$\ell_1 = 20$ 46.45%
SVM	Kernel/Parameters	Poly. - $q = 5$ 51.18%
		RBF - $\gamma = 0.5$ 51.18%
		Sigm. - $\gamma = 0.1$ 36.22%
CART	Min. samples/leaf	$\psi = 2$ 51.18%
		$\psi = 10$ 46.46%

Computational Experiment (cont.2)

- It is possible to observe parameter configurations where the SVM and CART methods yield the best results
- Similar performance is observed between the MLP and RBF network methods
- Taking the results obtained with the SVM method using the RBF kernel as reference, the class-wise accuracy percentages are:
 - Heavy rain: 25.0%
 - Ice melt: 83.33%
 - Torrential rain: 34.48%
 - Monsoon rain: 93.10%
 - Others: 35.14%
- Inherent characteristics of the data may explain the low accuracy rates observed for certain classes
- A possible solution to this issue is to merge the indicated classes into a single class

Non-linear Classifiers (RBF Net, SVM+Kernel and CART)

rogerio.negri@unesp.br